

Corporate Expense & Approval System

DeloitteEdge Consulting Ltd

Name: Abdul Mateen

Student ID: 20093908

Programme: Masters of Information Systems and Computing

Module: Programming for information Systems and Computing

Lecturer: Paul Laird

Assignment Title: Information System Design & Implementation

Date: 12 April 2026

Table of Contents

Section	Title
1	Introduction
2	Organisation & Problem Statement
3	System Requirements & Business Rules
4	Data Requirements & Storage
5	System Architecture
6	Implementation
7	Features
8	Testing
9	Tools & Technologies
10	Use of External Resources / AI
11	Challenges & Improvements
12	Dockerization & Deployment
13	Conclusion
14	References & Attributions
15	Appendix
16	How to Run the Project

1. Introduction

This project is a website built to solve the problem of manual corporate expense management and approval system. This is made for a fictional organization (DeloitteEdge Consulting Ltd) which has a lot of expenses and claims on daily basis. The app allows the employee to submit claims through the dashboard. The managers and finance officers then review and approve the claims through a multi step workflow. There is a Admin user which manages users, roles, departments, budgets. This App is made in Angular as frontend and NestJS as backend and MongoDB is used for datastorage. The app follows a server client model with REST Apis.

2. Organisation & Problem Statement

Organisation

DeloitteEdge is a mid sized professional services firm. It is modeled on a real consulting company Deloitte.

Current Problem / Inefficiency

The problem with current system used in the firm is that the expenses are managed manually. The records are stored in paper or spreadsheets. There is no audit trail available on who and when a claim is approved. The budget constraint are not strictly followed and most of the time there is over spending from allocated budget. The approval system is unstructured. There are constant issues with a submitted claim. Some time a claim duplicated or discarded at any stage without letting the employee know about it. At the end of a fiscal year when it is time to generate a report the data is all messed up.

Why an Information System Is Needed

There is need of a information system to centralize all expense submissions at one place. This will enforce structured multi step approval workflow. It will automatically track department budget utilisation. There will be role-specific dashboards and reporting. This system will reduce manual effort and human error and ensure accountability through clear approval records.

3. System Requirements & Business Rules

System Requirements

Functional Requirements

User Authentication & Access Control

- Users can log in with email and password
- System returns JWT token on successful login
- Each route protected based on user role
- Four roles: Employee, Manager, Finance Officer, Admin

CRUD - Expense Claims

- **Create:** Employee can create a new expense claim
- **Read:** Employee views own claims; Manager/Admin views all claims

- **Update:** Employee can edit a Draft claim (description, currency)
- **Delete:** Employee can delete a Draft claim; Admin can delete any claim

CRUD - Expense Items

- **Create:** Employee adds items to a Draft claim (category, amount, date, description)
- **Read:** Anyone with claim access can view its items
- **Update:** Employee can edit items on a Draft claim
- **Delete:** Employee can remove items from a Draft claim

CRUD - Users

- **Create:** Admin can create new users with assigned role and department
- **Read:** Admin/Manager can view all users, filter by role or status
- **Update:** Admin can update user details and role
- **Delete:** Admin can deactivate a user (soft delete)

CRUD - Departments

- **Create:** Admin can add new departments
- **Read:** All authenticated users can view departments
- **Update:** Admin can update department name, location, manager
- **Delete:** Not implemented - departments are deactivated via update

CRUD - Budgets

- **Create:** Admin creates a budget per department per fiscal year
- **Read:** Manager/Finance/Admin can view all budgets with utilisation
- **Update:** Admin can update total budget amount
- **Delete:** Not implemented

Workflow & Approvals

- Employee submits a claim
- Manager/Admin sets up approval steps
- Approvers approve or reject sequentially
- Employee cannot approve own claim
- Rejection halts entire workflow
- Final approval triggers automatic budget check

Dashboard

- Employee: own claim summary, spend by category
- Manager: pending approvals, system-wide claim counts
- Finance Officer: budget utilisation per department, monthly trends
- Admin: full system overview, top spenders, user counts

Non-Functional Requirements

Usability

- Role-aware navigation - users only see relevant menu items
- Status badges on claims for quick visual identification
- Form validation with clear error messages
- Demo account buttons on login for easy testing

Performance

- MongoDB indexes on frequently queried fields (employeeId, status, claimId)
- JWT authentication avoids repeated database lookups on every request

Security

- Passwords hashed using bcryptjs
- JWT tokens expire after 8 hours
- Refresh token available for session extension
- All routes protected - unauthenticated requests return 401
- Role violations return 403 with descriptive message

Validation & Integrity

- Global ValidationPipe with whitelist and forbidNonWhitelisted enabled
- All DTOs decorated with class-validator annotations
- Business rules enforced at service layer - not just database constraints
- Mongoose schemas enforce types, required fields, and unique indexes

Business Rules

BR01 - Claim Ownership

Each expense claim must be associated with exactly one employee. A claim cannot exist without an owner.

BR02 - Budget Must Be Positive

A department budget must have a total value greater than zero. Zero or negative budgets are not permitted.

BR03 - Approver Role Restriction

Only users with the role of Manager, Finance Officer, or Admin can be assigned as approvers in a workflow. Employees cannot be approvers.

BR04 - Minimum One Item Before Submission

A claim must have at least one expense item before it can be submitted. Submitting an empty claim is not permitted.

BR05 - Item Amount Must Be Positive

Each expense item must have an amount greater than zero. Zero or negative amounts are not permitted.

BR06 - Valid Expense Categories

Each expense item must belong to one of the defined categories: Travel, Accommodation, Meals, Equipment, Training, Subscriptions, Entertainment, or Other.

BR07 - Draft Claims Only Editable

Expense items and claim details can only be modified when the claim is in Draft status. Once submitted, the claim and its items are locked.

BR08 - No Self-Approval

An employee cannot be assigned as an approver on their own claim. The claim owner and all approvers must be different users.

BR09 - Minimum Two Approval Steps

Every approval workflow must have a minimum of two sequential steps. Single-step approvals are not permitted.

BR10 - Sequential Approval Enforcement

Approval steps must be completed in order. Step 2 cannot be actioned until Step 1 has been approved. Steps must be numbered sequentially starting from 1.

BR11 - Rejection Halts Workflow

If any approver rejects a claim at any step, the entire workflow is immediately halted. The claim status is set to Rejected and no further steps are processed.

BR12 - Budget Check on Final Approval

When the final approval step is completed, the system automatically checks whether the claim amount would exceed the remaining budget for the employee's department in the current fiscal year. If the budget would be exceeded, the approval is blocked.

BR13 - One Budget Per Department Per Year

Only one budget record can exist for a given department and fiscal year combination. Duplicate budgets for the same department and year are not permitted.

BR14 - Inactive Users Cannot Log In

A user account that has been deactivated cannot authenticate. Only active users can access the system.

BR15 - Attachment File Restrictions

Uploaded receipt files must be in one of the following formats: PDF, JPG, JPEG, or PNG. The maximum file size is 10MB. Files outside these restrictions are rejected.

4. Data Requirements & Storage

Entities & Fields

Users

Field	Type	Notes
<code>_id</code>	ObjectId	Auto-generated
<code>firstName</code>	String	Required
<code>lastName</code>	String	Required
<code>email</code>	String	Unique, lowercase
<code>passwordHash</code>	String	bcrypt hashed, never returned in responses
<code>jobTitle</code>	String	Required
<code>role</code>	Enum	employee, manager, finance_officer, admin
<code>departmentId</code>	ObjectId	Reference to Department
<code>isActive</code>	Boolean	Soft delete flag
<code>createdAt</code>	Date	Auto-generated
<code>updatedAt</code>	Date	Auto-generated

Departments

Field	Type	Notes
<code>_id</code>	ObjectId	Auto-generated
<code>departmentName</code>	String	Unique
<code>location</code>	String	Required
<code>managerName</code>	String	Required
<code>isActive</code>	Boolean	Default true
<code>createdAt</code>	Date	Auto-generated
<code>updatedAt</code>	Date	Auto-generated

Claims

Field	Type	Notes
<code>_id</code>	ObjectId	Auto-generated

Field	Type	Notes
employeeId	ObjectId	Reference to User
status	Enum	Draft, Submitted, Under Review, Approved, Rejected
submissionDate	Date	Set on submit
description	String	Required
totalAmount	Number	Auto-synced from items
currency	Enum	GBP, USD, EUR
createdAt	Date	Auto-generated
updatedAt	Date	Auto-generated

Items

Field	Type	Notes
_id	ObjectId	Auto-generated
claimId	ObjectId	Reference to Claim
category	Enum	Travel, Accommodation, Meals, Equipment, Training, Subscriptions, Entertainment, Other
amount	Number	Min 0.01
expenseDate	Date	Required
itemDescription	String	Required
receiptRequired	Boolean	Default true
createdAt	Date	Auto-generated
updatedAt	Date	Auto-generated

Workflows

Field	Type	Notes
_id	ObjectId	Auto-generated
claimId	ObjectId	Reference to Claim
approverId	ObjectId	Reference to User
stepNumber	Number	Sequential, min 1
decision	Enum	Pending, Approved, Rejected
decisionDate	Date	Set on decision
comments	String	Optional

Field	Type	Notes
createdAt	Date	Auto-generated
updatedAt	Date	Auto-generated

Budgets

Field	Type	Notes
_id	ObjectId	Auto-generated
departmentId	ObjectId	Reference to Department
fiscalYear	Number	Required
totalBudget	Number	Min 0.01
spentAmount	Number	Auto-updated on claim approval
createdAt	Date	Auto-generated
updatedAt	Date	Auto-generated

Attachments

Field	Type	Notes
_id	ObjectId	Auto-generated
claimId	ObjectId	Reference to Claim
fileName	String	Original file name
fileType	Enum	PDF, JPG, JPEG, PNG
fileSizeKB	Number	Max 10240 KB
filePath	String	Server file path
createdAt	Date	Auto-generated
updatedAt	Date	Auto-generated

Relationships

- A**Department** has many**Users** (one-to-many)
- A**Department** has many**Budgets** - one per fiscal year (one-to-many)
- A**User** has many**Claims** as an employee (one-to-many)
- A**Claim** has many**Items** (one-to-many)
- A**Claim** has many**Workflow steps** (one-to-many)
- Each**Workflow step** is assigned to one**User** as an approver (many-to-one)
- A**Claim** has many**Attachments** (one-to-many)

Storage Choice

For datastorage MongoDB is used. It is a document based NoSQL database. The data is stored as JSON files. The data from backend is modelled using Mongoose ODM. Indexes are used where ever required. Unique constraints for email, department name and other fields are added.

5. System Architecture

Overview

- Three-tier architecture: Frontend -> API -> Backend -> Database
- Frontend never communicates directly with the database
- All data access goes through the REST API
- No page refresh - Angular handles all routing client-side (SPA)

Frontend - Angular (Port 4200)

- Single Page Application built with Angular 21
- Standalone components - no NgModule required
- Angular Router handles all client-side navigation
- HTTP calls made via Angular's `HttpClient`
- JWT token attached to every request via a global HTTP interceptor
- Auth guard protects routes - unauthenticated users redirected to login
- Role-aware UI - menu items and pages shown based on logged-in user's role
- Proxy configuration forwards all `/api` requests to port 3000 during development

API Layer - NestJS REST API (Port 3000)

- Built with NestJS 10 using modular architecture
- Each feature is a self-contained module (auth, claims, items, workflow, etc.)
- Controllers handle routing and HTTP methods
- Services contain all business logic and database interaction
- Guards enforce authentication (`JwtAuthGuard`) and authorisation (`RolesGuard`)
- Global ValidationPipe validates all incoming request bodies
- Global exception filter returns consistent error response format
- JWT strategy validates token and loads user on every protected request

Database - MongoDB (Port 27017)

- Runs locally via Docker container
- Mongoose ODM used for schema definition and querying
- Each collection maps to a Mongoose schema with typed fields
- ObjectId references link documents across collections
- `.populate()` used to resolve references on query

Request Flow

1. User performs an action in Angular (e.g. submits a form)
2. Angular service makes an HTTP request with JWT in the `Authorization` header
3. NestJS receives the request - `JwtAuthGuard` validates the token
4. `RolesGuard` checks the user's role against the route's required roles

5. ValidationPipe validates the request body against the DTO
6. Controller passes the request to the service
7. Service applies business rules and queries MongoDB via Mongoose
8. Response is returned as JSON to Angular
9. Angular updates the DOM - no page reload

Security

- Passwords never stored in plain text - bcrypt hash
- JWT secret stored in `.env` - not hardcoded
- Token expires after 8 hours - refresh token valid for 7 days
- `passwordHash` field stripped from all API responses via Mongoose `toJSON` transform
- CORS configured to allow only trusted origins

6. Implementation

Backend

Project Structure

- NestJS application with 10 feature modules
- Each module contains: controller, service, schema, and DTOs
- Common folder contains shared guards, decorators, interceptors, and filters
- Entry point (`main.ts`) configures global pipe, exception filter, CORS, and API prefix

Authentication Module

- `POST /api/auth/login` - validates credentials, returns JWT + refresh token
- `POST /api/auth/refresh` - issues new access token using refresh token
- `GET /api/auth/me` - returns current logged-in user
- `PATCH /api/auth/change-password` - updates password after verifying current one
- Passport.js + passport-jwt strategy used for token validation
- JWT payload contains: `sub` (user ID), `email`, `role`

Claims Module

- `GET /api/claims` - returns claims (employees see own, managers/admin see all)
- `GET /api/claims/:id` - returns claim with items, workflow steps, and attachments
- `POST /api/claims` - creates a new Draft claim
- `PATCH /api/claims/:id` - updates description or currency (Draft only)
- `POST /api/claims/:id/submit` - submits claim
- `DELETE /api/claims/:id` - deletes claim and all related items, workflow, attachments

Items Module

- `GET /api/claims/:claimId/items` - returns all items for a claim
- `POST /api/claims/:claimId/items` - adds item to a Draft claim
- `PATCH /api/claims/:claimId/items/:itemId` - updates an item

- **DELETE** `/api/claims/:claimId/items/:itemId` - removes an item
- Adding or deleting an item automatically recalculates and updates **totalAmount** on the claim

Workflow Module

- **GET** `/api/workflow/pending` - returns pending approval steps for the logged-in approver
- **POST** `/api/workflow/:claimId/setup` - assigns approvers to sequential steps (Manager/Admin)
- **POST** `/api/workflow/:claimId/decide` - records Approved or Rejected decision with optional comment

Users Module

- **GET** `/api/users` - list all users with optional role and status filters
- **GET** `/api/users/:id` - get single user
- **POST** `/api/users` - create user (Admin only)
- **PATCH** `/api/users/:id` - update user details or role (Admin only)
- **DELETE** `/api/users/:id` - deactivate user, soft delete (Admin only)

Departments Module

- **GET** `/api/departments` - list all active departments
- **GET** `/api/departments/:id` - get single department
- **POST** `/api/departments` - create department (Admin only)
- **PATCH** `/api/departments/:id` - update department (Admin only)

Budgets Module

- **GET** `/api/budgets` - list budgets with optional fiscal year filter
- **POST** `/api/budgets` - create budget for a department and year (Admin only)
- **PATCH** `/api/budgets/:id` - update total budget amount (Admin only)

Attachments Module

- **POST** `/api/attachments/:claimId` - upload file (PDF, JPG, JPEG, PNG - max 10MB)
- **DELETE** `/api/attachments/:attachmentId` - delete file from disk and database
- Multer middleware handles file upload and storage

Dashboard Module

- **GET** `/api/dashboard/employee` - claim summary and spend by category for current user
- **GET** `/api/dashboard/manager` - pending approvals and system-wide claim stats
- **GET** `/api/dashboard/finance` - budget utilisation and monthly spend trends
- **GET** `/api/dashboard/admin` - full system overview including user counts and top spenders

Frontend

Project Structure

- Angular 20 standalone component architecture
- Core folder: models, services, guards, interceptors
- Pages folder: one component per page
- Single global stylesheet (`styles.css`) - no external CSS framework

Authentication Flow

- Login page collects email and password
- `AuthService.login()` calls `POST /api/auth/login`
- Token stored in `localStorage`
- `JwtInterceptor` attaches `Authorization: Bearer <token>` to every outgoing HTTP request
- `AuthGuard` checks for token before activating any protected route
- On logout, `localStorage` is cleared and user is redirected to login

Services

- `AuthService` - login, logout, token management, current user state
- `ClaimsService` - CRUD operations for claims
- `ItemsService` - CRUD operations for items within a claim
- `WorkflowService` - setup workflow, get pending, decide
- `UsersService` - user management
- `DepartmentsService` - department management
- `BudgetsService` - budget management
- `DashboardService` - role-specific dashboard data

Pages & User Interactions

Login

- User enters email and password and clicks Sign In
- Demo buttons auto-fill credentials for each role
- On success, redirected to Dashboard

Dashboard

- Loads automatically based on logged-in user's role
- Employee: sees own claim counts and spend by category
- Manager: sees pending approvals with links to review
- Finance Officer: sees budget utilisation with progress bars per department
- Admin: sees system-wide stats, user counts, and top spenders

Claims

- User sees list of claims filtered by status dropdown
- Create form appears inline - no page navigation required
- Submit and Delete buttons appear conditionally based on claim status
- Clicking a claim navigates to the Claim Detail page

Claim Detail / Items

- Shows claim summary (description, total, currency, status)
- Add Item form appears inline with category, amount, date, description fields
- Edit and Delete buttons on each item row
- Total amount updates automatically after each add/edit/delete
- Workflow history shown at the bottom of the page

Workflow

- Manager sees list of submitted claims and sets up approval steps
- Approver selects decision (Approve/Reject) and adds optional comment
- Pending approvals listed with claim details and employee name

Users

- Admin sees full user list with role and status filters
- Create and Edit forms appear inline
- Deactivate button performs soft delete

Departments

- Admin sees all departments in a table
- Create and Edit forms appear inline

Budgets

- Shows summary stats (total allocated, spent, remaining, overall %)
- Progress bar per department showing utilisation
- Fiscal year filter dropdown
- Admin can create new budgets or update existing ones

7. Features

Core Features

CRUD - Expense Claims

- Create new expense claim with description and currency
- Read claims - employees see own, managers and admins see all
- Filter claims by status (Draft, Submitted, Under Review, Approved, Rejected)
- Update claim description and currency (Draft status only)
- Submit claim - triggers validation (must have at least one item)
- Delete claim - removes claim and all related items, workflow steps, and attachments

CRUD - Expense Items

- Add expense items to a Draft claim
- Eight expense categories: Travel, Accommodation, Meals, Equipment, Training, Subscriptions, Entertainment, Other
- Edit item amount, date, category, and description
- Delete individual items

- Claim total amount automatically recalculated on every add, edit, or delete

CRUD - Users (Admin)

- Create new users with role and department assignment
- View all users with role and active status filters
- Update user details, job title, and role
- Deactivate users (soft delete - data preserved)

CRUD - Departments (Admin)

- Create new departments with name, location, and manager
- View all active departments
- Update department details

CRUD - Budgets (Admin/Finance)

- Create annual budget per department
 - View budgets with fiscal year filter
 - Update total budget amount
 - Budget automatically decremented on claim approval
-

Additional Features

Multi-Step Approval Workflow

- Manager assigns minimum 2 sequential approvers to a submitted claim
- Each approver can approve or reject with optional comments
- Steps enforced in order - step 2 blocked until step 1 is complete
- Rejection immediately halts entire workflow and marks claim as Rejected
- Final approval triggers automatic budget check against department budget

Role-Based Access Control

- Four roles with different permissions: Employee, Manager, Finance Officer, Admin
- Route-level protection via NestJS RolesGuard
- UI-level protection - menu items and buttons shown based on role
- Employees can only see and interact with their own claims

Role-Aware Dashboard

- Each role gets a completely different dashboard view
- Employee: claim summary, approved spend this year, spend by category
- Manager: pending approvals count, recent submissions needing attention
- Finance Officer: budget utilisation per department with progress bars
- Admin: system-wide stats, user breakdown by role, top spenders

Reporting

- Spending by department with budget vs actual comparison
- Spending by employee ranked by total approved amount
- Spending by expense category with item counts
- Monthly trends showing approved, rejected, and pending claims per month
- Year selector to switch between fiscal years

Validation

- Server-side: class-validator on all DTOs with whitelist enabled
- Business rule validation: 15 rules enforced at service layer
- Client-side: required field checks before API calls are made
- Descriptive error messages returned to the frontend on failure

File Attachments

- Upload receipts against a claim (PDF, JPG, JPEG, PNG)
- File size limit of 10MB enforced
- Files stored on server disk via Multer middleware
- Attachments deleted from disk when removed

Seeded Demo Data

- 6 departments, 12 users, 6 budgets
- Claims in all statuses (Draft, Submitted, Under Review, Approved, Rejected)
- Pre-configured workflow steps and items
- All demo accounts use Password123! for easy testing

API Testing Collection

- Bruno collection with 47 requests covering all endpoints
- Environment variables for baseUrl, token, and entity IDs
- Post-response scripts automatically save JWT token and entity IDs
- Separate login requests for each role

8. Testing

Overview

- Unit tests written using Jest and ts-jest
- Tests run at the service layer - no real database connection required
- Mongoose models mocked using Jest mock functions
- NestJS `TestingModule` used to bootstrap each service in isolation
- Each spec file follows: mock setup → `beforeEach` reset → grouped `describe` blocks per method

Unit Tests - Auth Service

File: `src/auth/auth.service.spec.ts`

login()

Test	Expected Result
Valid email and password	Returns JWT token, refresh token, and user object
User not found	Throws <code>UnauthorizedException</code>
User account is inactive	Throws <code>UnauthorizedException</code>
Wrong password	Throws <code>UnauthorizedException</code>
Successful login	<code>passwordHash</code> is not present in the response

refresh()

Test	Expected Result
Valid refresh token	Returns new access token
Invalid refresh token	Throws <code>UnauthorizedException</code>
Expired refresh token	Throws <code>UnauthorizedException</code>

changePassword()

Test	Expected Result
Correct current password and valid new password	Password updated, <code>save()</code> called
Wrong current password	Throws <code>UnauthorizedException</code>
New password under 8 characters	Throws <code>BadRequestException</code>
User not found	Throws <code>UnauthorizedException</code>

Unit Tests - Claims Service

File: `src/claims/claims.service.spec.ts`

findAll()

Test	Expected Result
Employee role	Query includes <code>employeeId</code> filter - only own claims returned
Manager role	Query has no <code>employeeId</code> filter - all claims returned
Status filter provided	Query includes <code>status</code> field
Any role	Each result includes <code>itemCount</code> and <code>attachmentCount</code>

findById()

Test	Expected Result
Valid claim ID, own claim	Returns claim with items, workflow, and attachments
Claim does not exist	Throws <code>NotFoundException</code>
Employee viewing another employee's claim	Throws <code>ForbiddenException</code>

`create()`

Test	Expected Result
Valid input	Claim created with <code>status: Draft</code> , <code>totalAmount: 0</code> , <code>employeeId</code> set

`update()`

Test	Expected Result
Valid update on Draft claim	<code>save()</code> called with updated fields
Claim does not exist	Throws <code>NotFoundException</code>
Claim is not Draft	Throws <code>UnprocessableEntityException</code>
User does not own the claim	Throws <code>ForbiddenException</code>

`submit()`

Test	Expected Result
Draft claim with items	Status changes to <code>Submitted</code> , <code>save()</code> called
Draft claim with no items	Throws <code>UnprocessableEntityException</code>
Claim is not Draft	Throws <code>UnprocessableEntityException</code>
Claim does not exist	Throws <code>NotFoundException</code>
User does not own the claim	Throws <code>ForbiddenException</code>

`delete()`

Test	Expected Result
Draft claim owned by user	Claim, items, workflow, and attachments all deleted
Claim does not exist	Throws <code>NotFoundException</code>
Non-admin deleting a Submitted claim	Throws <code>UnprocessableEntityException</code>
Admin deleting a non-Draft claim	Deletion succeeds

Running the Tests

```
# Run all tests
npm test

# Run a single spec file
npm test -- auth.service.spec
npm test -- claims.service.spec
```

9. Tools & Technologies

Runtime & Language

Tool	Version	Purpose
Node.js	v22	Server-side JavaScript runtime
TypeScript	~5.4	Strongly typed superset of JavaScript

Backend

Tool	Version	Purpose
NestJS	^10.3	Backend framework - modular, decorator-based
Mongoose	^8.4	MongoDB ODM - schema definition and querying
Passport.js	^0.7	Authentication middleware
passport-jwt	^4.0	JWT strategy for Passport
@nestjs/jwt	^10.2	JWT signing and verification
bcryptjs	^2.4	Password hashing
class-validator	^0.14	DTO validation decorators
class-transformer	^0.5	Type transformation for request bodies
Multer	^1.4	File upload handling

Frontend

Tool	Version	Purpose
Angular	^20	SPA framework - components, routing, HTTP
RxJS	~7.8	Reactive programming - HTTP observables
Angular Router	^20	Client-side routing
Angular HttpClient	^20	HTTP requests to the API

Database

Tool	Version	Purpose
MongoDB	7 (Docker)	NoSQL document database
Docker	Latest	Runs MongoDB in an isolated container

Development & Testing Tools

Tool	Purpose
Bruno	API testing - 47 requests covering all endpoints
ts-node	Runs TypeScript directly - used for seed script
npm	Package management

Version Control

Tool	Purpose
Git	Version control
GitHub	Remote repository and code hosting

Why These Tools

NestJS over Express

- Opinionated structure enforces separation of concerns
- Built-in dependency injection, guards, interceptors, and pipes
- TypeScript-first - catches errors at compile time
- Modular architecture scales well as the project grows

MongoDB over SQL

- Flexible schema suited to evolving requirements
- JSON-like documents map naturally to TypeScript objects
- Easy to run locally via Docker with minimal configuration
- Mongoose provides validation, virtuals, and type safety

Angular over React/Vue

- Full framework - routing, HTTP, forms all built in
- Strong TypeScript integration
- Standalone component architecture (Angular 20) reduces boilerplate
- HttpClient interceptors make JWT injection clean and centralised

JWT over Session-Based Auth

- Stateless - no server-side session storage required

- Token carries role information - reduces database lookups
- Works well with a REST API architecture
- Refresh token pattern extends session without re-login

Bruno over Postman

- Stores collections as plain `.bru` text files - version control friendly
- No account required - fully local
- Lightweight and fast
- Post-response scripts allow automatic token and ID saving

10. Use of External Resources / AI

Frameworks & Libraries

Resource	Source	What It Provided	What Was Built On Top
NestJS	nestjs.com	Module system, decorators, guards, pipes	All business logic, custom guards, DTOs, service layer
Mongoose	mongoosejs.com	Schema definition, querying, populate	All schemas, indexes, virtuals, relationships
Angular 20	angular.dev	Component framework, routing, HttpClient	All components, services, interceptors, guards
passport-jwt	npmjs.com	JWT extraction and verification strategy	Custom strategy wired to user lookup and role loading
bcryptjs	npmjs.com	Password hashing function	Integrated into auth service for login and password change
class-validator	npmjs.com	Validation decorators	Applied to all DTOs across every module
Multer	npmjs.com	File upload middleware	Configured with custom file filter, size limits, disk storage
RxJS	rxjs.dev	Observable streams	Used in Angular services and components for HTTP calls

AI Usage

This project was developed with the assistance of Claude (Anthropic) as a co-pilot tool, operating at **Level 3** of the institution's Generative AI Assessment Scale - AI-ASSISTED EDITING.

What AI Was Used For

- Generating boilerplate NestJS module, controller, and service files
- Scaffolding Angular standalone components and services
- Generating the Bruno API testing collection

- Suggesting fixes for TypeScript compilation errors
- Writing the seed script for demo data

How AI Output Was Used

- All AI-generated code was reviewed before use
- Several files required manual correction after generation
 - `audit.module.ts` - controller and module were incorrectly merged into one file
 - Workflow DTOs were missing class-validator decorators causing 400 errors
 - ObjectId comparison bugs in `WorkflowService` required manual debugging and fixing
 - `preserveNullAndEmpty` typo in reports aggregation pipeline required correction
- AI was used as a productivity tool - not a replacement for understanding
- All generated code was tested, debugged, and modified where necessary

Attribution Summary

Resource	Licence	Link
NestJS	MIT	github.com/nestjs/nest
Mongoose	MIT	github.com/Automattic/mongoose
Angular	MIT	github.com/angular/angular
passport-jwt	MIT	github.com/mikenicholson/passport-jwt
bcryptjs	MIT	github.com/dcodeIO/bcrypt.js
class-validator	MIT	github.com/typestack/class-validator
class-transformer	MIT	github.com/typestack/class-transformer
Multer	MIT	github.com/expressjs/multer
RxJS	Apache-2.0	github.com/ReactiveX/rxjs
Claude (Anthropic)	N/A - AI tool	anthropic.com

11. Challenges & Improvements

Challenges Faced

1. ObjectId Type Mismatch in Workflow Queries

- Mongoose `.lean()` returns `_id` as a raw ObjectId object, not a string
- Comparing `currentUser._id` directly against stored `approverId` failed silently
- Query returned no results - caused "previous step not completed" error even when it was
- **Fix:** Wrapped all ObjectId comparisons with `new Types.ObjectId(id.toString())`

2. Docker Networking Issue

- Initial project used MSSQL with Docker

- Init container used `localhost` as hostname - could not reach the SQL Server container
 - Containers on the same Docker network must reference each other by service name
 - **Fix:** Changed hostname from `localhost` to `deloitteedge-sqlserver`
-

What Would Be Improved With More Time

Technical Improvements

- Add unit and integration tests with Jest and Supertest
- Implement refresh token rotation - invalidate old refresh tokens on use
- Add pagination to claims and users list endpoints
- Add sorting options to all list views (by date, amount, status)
- Implement full audit logging - write to `auditlogs` collection on every create, update, delete
- Add email notifications on claim submission and approval decision
- Implement file serving endpoint so uploaded receipts can be viewed in the browser
- Add search functionality to claims list (by description or employee name)

Frontend Improvements

- Add form validation feedback inline rather than alert-style messages
- Add loading skeletons instead of plain "Loading..." text
- Make dashboard charts visual - bar or pie charts for spend data
- Add confirmation modals instead of browser `confirm()` dialogs
- Make the UI fully responsive for mobile screens

Architecture Improvements

- Move to a microservices architecture for larger scale
- Add CI/CD pipeline with GitHub Actions for automated testing and deployment

12. Dockerization & Deployment

Overview

- Full stack application containerised using Docker and Docker Compose
 - Three containers: MongoDB database, NestJS backend, Angular frontend
 - All containers communicate over a dedicated Docker bridge network
 - Application deployed to a Microsoft Azure Virtual Machine
 - Live URL: <http://testmateen.norwayeast.cloudapp.azure.com:8080/>
-

Container Architecture

Container	Image	Port	Purpose
<code>deloitteedge-mongodb</code>	<code>mongo:7-jammy</code>	27017	MongoDB database
<code>deloitteedge-backend</code>	Custom (Node.js 24)	3000	NestJS REST API

Container	Image	Port	Purpose
<code>deloitteedge-frontend</code>	Custom (Nginx)	8080	Angular SPA served via Nginx

All three containers are connected via a custom bridge network called `deloitteedge-network`, allowing them to reference each other by container name rather than IP address.

Backend Dockerfile

The backend uses a single-stage Node.js 24 image:

- Copies `package.json` and installs dependencies via `npm install`
 - Copies the full source and runs `npm run build` to compile TypeScript
 - Exposes port 3000 and starts the app with `npm run start:prod`
 - The NestJS production build runs the compiled JavaScript from the `dist/` folder
-

Frontend Dockerfile

The frontend uses a two-stage build:

Stage 1 - Builder:

- Node.js 24 base image
- Uses `npm ci` with Docker layer caching (`--mount=type=cache`) for faster rebuilds
- Runs `npm run build` to produce the compiled Angular static files

Stage 2 - Runner:

- Nginx base image
 - Copies the compiled static files from the builder stage into Nginx's HTML directory
 - Uses a custom `nginx.conf` for SPA routing and API proxying
 - Runs as the `nginx` non-root user for security
 - Exposes port 8080
-

Docker Compose

The `docker-compose.yml` at the project root orchestrates all three services:

- **MongoDB** starts first with a healthcheck - backend waits until MongoDB responds to `db.adminCommand('ping')` before starting
- **Backend** depends on MongoDB being healthy (`condition: service_healthy`) - prevents startup failures if the database is not yet ready
- **Frontend** depends on the backend starting and proxies all `/api` requests to `http://backend:3000` via Nginx
- A named volume `mongo_data` persists database data across container restarts
- A named volume `uploads_data` persists uploaded receipt files across container restarts
- Environment variables for JWT secrets, MongoDB URI, and CORS origin are configured directly in the compose file

Nginx Configuration

Nginx serves the Angular build and handles two responsibilities:

- **Static file serving** - serves the compiled Angular app from `/usr/share/nginx/html`
- **SPA routing** - all routes fall back to `index.html` so Angular's client-side router handles navigation without 404 errors
- **API proxy** - requests to `/api` are forwarded to the backend container at `http://backend:3000`

This means the frontend and API are both accessible on the same port (8080) from the outside, with Nginx acting as a reverse proxy.

Deployment

The application is deployed on a Microsoft Azure Virtual Machine in the Norway East region:

- Docker and Docker Compose installed on the VM
- Project files copied to the VM via Git
- Started with a single command from the project root:

```
docker compose up -d
```

- The `--detach` flag runs all containers in the background
- Containers are configured with `restart: unless-stopped` - they automatically restart after a VM reboot or crash

Live application: <http://testmateen.norwayeast.cloudapp.azure.com:8080/>

13. Conclusion

- Built a fully functional Corporate Expense & Approval System for DeloitteEdge Consulting Ltd
- System meets the core assignment requirements:
 - REST API architecture with frontend making API calls - no page refresh
 - Full CRUD operations across claims, items, users, departments, and budgets
 - JavaScript-based stack (TypeScript/Node.js backend, Angular frontend)
 - NoSQL database (MongoDB) with structured schema and relationships
- 15 business rules implemented and enforced at the service layer
- Multi-step sequential approval workflow with budget validation on final approval
- Role-based access control across four roles - each with different data access and permissions
- Role-aware dashboard and reporting system providing real-time visibility
- Database seeded with realistic demo data across all collections
- API fully tested end-to-end using a Bruno collection of 47 requests
- Frontend delivers a clean, usable interface with minimal styling as per requirements
- All external libraries and AI usage clearly attributed
- Known limitations acknowledged - testing section to be completed, some improvements identified for future development

14. References & Attributions

Frameworks & Libraries

- NestJS - MIT Licence - <https://github.com/nestjs/nest>
- Mongoose - MIT Licence - <https://github.com/Automattic/mongoose>
- Angular - MIT Licence - <https://github.com/angular/angular>
- Passport.js - MIT Licence - <https://github.com/jaredhanson/passport>
- passport-jwt - MIT Licence - <https://github.com/mikenicholson/passport-jwt>
- @nestjs/jwt - MIT Licence - <https://github.com/nestjs/jwt>
- bcryptjs - MIT Licence - <https://github.com/dcodeIO/bcrypt.js>
- class-validator - MIT Licence - <https://github.com/typestack/class-validator>
- class-transformer - MIT Licence - <https://github.com/typestack/class-transformer>
- Multer - MIT Licence - <https://github.com/expressjs/multer>
- RxJS - Apache-2.0 Licence - <https://github.com/ReactiveX/rxjs>
- zone.js - MIT Licence - <https://github.com/angular/angular/tree/main/packages/zone.js>

Documentation & Learning Resources

- NestJS Official Documentation - <https://docs.nestjs.com>
- Angular Official Documentation - <https://angular.dev>
- Mongoose Official Documentation - <https://mongoosejs.com/docs>
- MongoDB Manual - <https://www.mongodb.com/docs/manual>
- TypeScript Handbook - <https://www.typescriptlang.org/docs>
- JWT Introduction - <https://jwt.io/introduction>
- class-validator Documentation - <https://github.com/typestack/class-validator#readme>
- Bruno Documentation - <https://docs.usebruno.com>

15. Appendix

A - API Endpoint Summary

Method	Endpoint	Access	Description
POST	/api/auth/login	Public	Login and receive JWT
POST	/api/auth/refresh	Public	Refresh access token
GET	/api/auth/me	Any	Get current user
PATCH	/api/auth/change-password	Any	Change own password
GET	/api/claims	Any	List claims
POST	/api/claims	Any	Create claim
GET	/api/claims/:id	Any	Get claim detail
PATCH	/api/claims/:id	Any	Update claim
POST	/api/claims/:id/submit	Any	Submit claim

Method	Endpoint	Access	Description
DELETE	/api/claims/:id	Any	Delete claim
GET	/api/claims/:id/items	Any	List items
POST	/api/claims/:id/items	Any	Add item
PATCH	/api/claims/:id/items/:itemId	Any	Update item
DELETE	/api/claims/:id/items/:itemId	Any	Delete item
GET	/api/workflow/pending	Manager+	Get pending approvals
POST	/api/workflow/:id/setup	Manager+	Setup approval steps
POST	/api/workflow/:id/decide	Manager+	Approve or reject
GET	/api/users	Manager+	List users
POST	/api/users	Admin	Create user
GET	/api/users/:id	Manager+	Get user
PATCH	/api/users/:id	Admin	Update user
DELETE	/api/users/:id	Admin	Deactivate user
GET	/api/departments	Any	List departments
POST	/api/departments	Admin	Create department
GET	/api/departments/:id	Any	Get department
PATCH	/api/departments/:id	Admin	Update department
GET	/api/budgets	Manager+	List budgets
POST	/api/budgets	Admin	Create budget
PATCH	/api/budgets/:id	Admin	Update budget
POST	/api/attachments/:claimId	Any	Upload attachment
DELETE	/api/attachments/:id	Any	Delete attachment
GET	/api/dashboard/employee	Any	Employee dashboard
GET	/api/dashboard/manager	Manager+	Manager dashboard
GET	/api/dashboard/finance	Finance+	Finance dashboard
GET	/api/dashboard/admin	Admin	Admin dashboard

B - Demo Accounts

Name	Email	Role	Password
Tom Bradley	tom.bradley@deloitteedge.co.uk	Employee	Password123!

Name	Email	Role	Password
Sarah Mitchell	sarah.mitchell@deloitteedge.co.uk	Manager	Password123!
Michael Hartley	michael.hartley@deloitteedge.co.uk	Finance Officer	Password123!
Admin User	admin@deloitteedge.co.uk	Admin	Password123!

16. How to Run the Project

Prerequisites

- Docker installed
- No other tools required

Setup

Clone or download the project so the folder structure looks like this:

```
project-root/  
├─ docker-compose.yml  
├─ backend/  
└─ frontend/
```

Start Everything

From the project root folder, run:

```
docker compose up -d
```

This starts four containers:

- `deloitteedge-mongodb` - MongoDB database on port `27017`
- `deloitteedge-backend` - NestJS REST API on port `3000`
- `deloitteedge-frontend` - Angular app served via nginx on port `4200`

Open the App

```
http://localhost:8080
```

Seed Data

To seed some data in the project run this command

This will remove all the current data stored in database

```
docker exec -it deloitteedge-backend npx ts-node /usr/src/app/src/seed.ts
```

Demo Accounts (all use Password123!)

Name	Email	Role
Tom Bradley	tom.bradley@deloitteedge.co.uk	Employee
Sarah Mitchell	sarah.mitchell@deloitteedge.co.uk	Manager
Michael Hartley	michael.hartley@deloitteedge.co.uk	Finance Officer
Admin User	admin@deloitteedge.co.uk	Admin

Stop the Project

```
docker compose down
```

Commit by Commit References

- Commit-1 :
 - [Node.js Documentation](#)
 - [Express.js Documentation](#)
- Commit-2 :
 - [NestJS Documentation](#)
- Commit-3 :
 - [NestJs Documentation \(Exception Filters\)](#)
 - [Docker Compose Documentation](#)
- Commit 4:

- [JWT Authentication Documentation](#)
 - [NestJS Authentication Documentation](#)
- Commit 5:
 - [NestJS Authorization Documentation](#)
 - [Role-Based Access Control \(RBAC\) Concepts](#)
 - Github Copilot Chat - help with implementing role-based access control in NestJS
- Commit 6:
 - [NestJS Documentation \(Controllers & Providers\)](#)
- Commit 39
 - [Angular Documentation \(Guards & Interceptors\)](#)
 - [RxJS Documentation \(BehaviorSubject\)](#)
- Commit 47
 - [Docker Documentation \(Multi-stage Builds & Networking\)](#)
 - [Nginx Guide \(Reverse Proxy & Load Balancing\)](#)
- Commit 49:
 - [Nginx Configuration for Angular SPA](#)
 - [Docker Compose Documentation \(Healthchecks\)](#)